

Математические основы алгоритмов

Конспект по 26 экзаменационным билетам

Весенний семестр 2026 года

Конспект построен по лекциям А. В. Тискина и расширен полными формулировками, доказательствами, алгоритмами и оценками трудоёмкости. Для каждого билета выделены ключевая идея, основной материал и каркас устного ответа.

Подготовлено для Тимура Саара

Содержание

1	Первообразные корни, DFT, свёртки и умножение полиномов	3
1.1	Первообразный корень из единицы	3
1.2	Прямое и обратное DFT	3
1.3	Три вида свёртки	3
1.4	Карацуба и DFT-умножение	4
2	Быстрое преобразование Фурье	5
2.1	FFT с прореживанием по времени (DIT)	5
2.2	FFT с прореживанием по частоте (DIF)	5
2.3	Общая шестиэтапная конструкция	6
3	Вычислительные схемы и сети сортировки	6
3.1	Модель схем	6
3.2	Принцип нулей-единиц	6
3.3	Нечётно-чётное слияние Бэтчера	7
3.4	Битонное слияние	7
4	Префиксное накопление, схема Ладнера–Фишера и BSP	7
4.1	Задача prefix scan	8
4.2	Схема Ладнера–Фишера	8
4.3	Модель BSP	8
4.4	Ладнер–Фишер на BSP	8
5	Линейные рекурренты и параллельное сложение	9
5.1	Обобщённая линейная рекуррента	9
5.2	Сложение двоичных чисел	10
6	Линейное программирование и кратчайшие пути	10
6.1	Формы LP	10
6.2	Идея симплекс-метода	11
6.3	Кратчайшие пути как LP	11
7	Максимальный поток и алгоритм Форда–Фалкерсона	11
7.1	Остаточная сеть	12
7.2	Форд–Фалкерсон	12
8	Двойственная LP, дополняющая нежёсткость и слабая двойственность	13
8.1	Дополняющая нежёсткость	13
9	Сильная теорема двойственности	14
9.1	Геометрическое доказательство в невырожденном случае	14
10	Дробный минимальный разрез как двойственная задача	15
11	Приближённые алгоритмы и вершинное покрытие	16
11.1	Определение	16
11.2	Невзвешенное вершинное покрытие	16
11.3	Взвешенное покрытие через LP	17
12	Метод двойственной допустимости для weighted vertex cover	17

13 Метрическая TSP: двойное дерево и Кристофидес–Сердюков	18
13.1 Алгоритм двойного дерева	18
13.2 Алгоритм Кристофидеса–Сердюкова	18
14 Алгоритм Фрейвальдса	19
15 Проверка равенства многочленов и лемма Шварца–Зиппеля	20
16 Универсальное хеширование и семейство multiply-add	20
17 Семейство multiply-shift	21
18 Совершенное и двухуровневое хеширование	22
18.1 Один уровень	22
18.2 Два уровня FKS	22
19 Разрешение коллизий: списки и линейное зондирование	23
19.1 Хеширование списками	23
19.2 Линейное зондирование	23
20 Бинарное зондирование	24
21 SAT, k-SAT и детерминированный алгоритм для 2-SAT	25
22 Вероятностный алгоритм для 2-SAT	26
23 Вероятностный алгоритм для 3-SAT	26
24 Алгоритм Миллера–Рабина	27
25 Онлайн-алгоритмы: аренда и кеширование	28
25.1 Аренда самоката	28
25.2 Кеширование	28
26 Минимальное остовное дерево: Борувка, Прим и Краскал	29
26.1 Алгоритм Борувки	30
26.2 Алгоритм Ярника–Прима	30
26.3 Алгоритм Краскала	30
Сводная таблица оценок	31
Источник и границы конспекта	31

Билет 1. Первообразные корни, DFT, свёртки и умножение полиномов

Ключевая идея

DFT переводит коэффициенты полинома в его значения на циклической группе корней из единицы. В этом представлении свёртка превращается в покомпонентное умножение. Обратное DFT выполняет интерполяцию.

1.1 Первообразный корень из единицы

Пусть R — коммутативное кольцо с единицей, $n \geq 2$. Элемент $\omega \in R$ называется *первообразным корнем степени n из единицы*, если

$$\omega^n = 1, \quad \omega^k \neq 1 \quad (1 \leq k < n).$$

Тогда $\langle \omega \rangle = \{1, \omega, \dots, \omega^{n-1}\}$ — циклическая группа порядка n . В $\mathbb{C}$ можно взять $\omega = e^{2\pi i/n}$; вообще $e^{2\pi i k/n}$ первообразен тогда и только тогда, когда $\gcd(k, n) = 1$.

Для обратимости DFT требуется, чтобы $n \cdot 1_R$ был обратим в R . В доказательстве ортогональности также используется

$$\sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n, & k \equiv 0 \pmod{n}, \\ 0, & k \not\equiv 0 \pmod{n}. \end{cases}$$

При $k \not\equiv 0$ это следует из $(\omega^k - 1) \sum_{j=0}^{n-1} \omega^{kj} = \omega^{kn} - 1 = 0$ и отсутствия делителей нуля для $\omega^k - 1$.

1.2 Прямое и обратное DFT

Для $a = (a_0, \dots, a_{n-1})^T$ определим

$$\hat{a}_k = \sum_{j=0}^{n-1} a_j \omega^{kj}, \quad 0 \leq k < n.$$

Матрично $\hat{a} = F_{n,\omega} a$, где $(F_{n,\omega})_{kj} = \omega^{kj}$.

Теорема 1.1 (Обращение DFT).

$$F_{n,\omega^{-1}} F_{n,\omega} = nI, \quad a_j = \frac{1}{n} \sum_{k=0}^{n-1} \hat{a}_k \omega^{-kj}.$$

Доказательство. Элемент (r, j) произведения матриц равен

$$\sum_{k=0}^{n-1} \omega^{-rk} \omega^{kj} = \sum_{k=0}^{n-1} \omega^{(j-r)k}.$$

Сумма равна n при $j = r$ и 0 иначе. Следовательно, произведение равно nI . $\square$

Если $a(x) = \sum_{j=0}^{n-1} a_j x^j$, то $\hat{a}_k = a(\omega^k)$. Поэтому DFT — вычисление значений, а обратное DFT — интерполяция на $\langle \omega \rangle$.

1.3 Три вида свёртки

Пусть $a, b \in R^n$.

Линейная свёртка.

$$(a * b)_k = \sum_{i+j=k} a_i b_j, \quad 0 \leq k \leq 2n - 2.$$

Это коэффициенты обычного произведения $a(x)b(x)$. Если выбрать $N \geq 2n - 1$, дополнить оба вектора нулями до длины N и взять первообразный корень ω степени N , то

$$a * b = \text{IDFT}_N(\text{DFT}_N(a) \circ \text{DFT}_N(b)),$$

где $\circ$ — покоординатное умножение.

Циклическая свёртка. Она соответствует умножению по модулю $x^n - 1$:

$$(a \circledast b)_k = \sum_{i=0}^{n-1} a_i b_{(k-i) \bmod n}.$$

Поскольку $(\omega^j)^n = 1$, вычисление значений согласовано с фактор-кольцом $R[x]/(x^n - 1)$, и

$$a \circledast b = \frac{1}{n} F_{n, \omega^{-1}}((F_{n, \omega} a) \circ (F_{n, \omega} b)).$$

Косоциклическая свёртка. Она соответствует умножению по модулю $x^n + 1$:

$$(a \circledast_- b)_k = \sum_{i=0}^k a_i b_{k-i} - \sum_{i=k+1}^{n-1} a_i b_{n+k-i}.$$

Пусть ψ — первообразный корень степени $2n$, $\omega = \psi^2$, и $D_\psi = \text{diag}(1, \psi, \dots, \psi^{n-1})$. Подстановка $x = \psi y$ переводит $x^n + 1$ в константный множитель от $y^n - 1$, поэтому

$$a \circledast_- b = D_\psi^{-1} \text{IDFT}_{n, \omega}(\text{DFT}_{n, \omega}(D_\psi a) \circ \text{DFT}_{n, \omega}(D_\psi b)).$$

1.4 Карацуба и DFT-умножение

Пусть $n = 2m$, $a = a_0 + x^m a_1$, $b = b_0 + x^m b_1$, где части имеют длину m . Тогда

$$ab = z_0 + x^m(z_1 - z_0 - z_2) + x^{2m} z_2,$$

где

$$z_0 = a_0 b_0, \quad z_2 = a_1 b_1, \quad z_1 = (a_0 + a_1)(b_0 + b_1).$$

Используются три рекурсивных умножения вместо четырёх:

$$T(n) = 3T(n/2) + \mathcal{O}(n) = \mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585}).$$

DFT-метод выбирает N — степень двойки, $2n - 1 \leq N < 4n$, вычисляет значения a, b на корнях степени N , перемножает их и интерполирует. При FFT:

$$T(n) = \mathcal{O}(n \log n)$$

арифметических операций в подходящем поле или кольце.

Типичная ошибка

Для линейной свёртки нельзя брать DFT длины n : старшие коэффициенты “завернутся” и получится циклическая свёртка. Нужна длина не меньше $2n - 1$.

Каркас ответа на экзамене

Определить первообразный корень; записать прямое и обратное DFT и доказать обращение геометрической суммой; объяснить “значения вместо коэффициентов”; выписать линейную, циклическую и косоциклическую свёртки; завершить Карацубой $\mathcal{O}(n^{\log_2 3})$ и DFT-умножением $\mathcal{O}(n \log n)$.

Билет 2. Быстрое преобразование Фурье**Ключевая идея**

FFT использует факторизацию размера DFT и повторное применение малых DFT. Для $n = 2^r$ каждый уровень выполняет $\mathcal{O}(n)$ операций, а уровней $\log_2 n$.

2.1 FFT с прореживанием по времени (DIT)

Пусть n чётно и ω — корень степени n . Разделим коэффициенты на чётные и нечётные:

$$A_0(y) = \sum_{j=0}^{n/2-1} a_{2j} y^j, \quad A_1(y) = \sum_{j=0}^{n/2-1} a_{2j+1} y^j,$$

так что $A(x) = A_0(x^2) + x A_1(x^2)$. Для $0 \leq k < n/2$:

$$\hat{a}_k = E_k + \omega^k O_k, \quad \hat{a}_{k+n/2} = E_k - \omega^k O_k,$$

где $E = \text{DFT}_{n/2, \omega^2}(a_0, a_2, \dots)$ и аналогично O для нечётных коэффициентов. Пара

$$(E_k, O_k) \mapsto (E_k + \omega^k O_k, E_k - \omega^k O_k)$$

называется бабочкой.

Алгоритм

1. Рекурсивно вычислить DFT чётной и нечётной подпоследовательностей.
2. Для $k = 0, \dots, n/2 - 1$ выполнить одну бабочку.
3. База: $n = 1$ или $n = 2$.

Рекуррентность:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Итеративная реализация обычно требует бит-реверсивной перестановки входов.

2.2 FFT с прореживанием по частоте (DIF)

Теперь сначала объединяем пары входов $a_j, a_{j+n/2}$:

$$u_j = a_j + a_{j+n/2}, \quad v_j = (a_j - a_{j+n/2}) \omega^j.$$

Тогда чётные частоты являются DFT вектора u , а нечётные — DFT вектора v относительно корня ω^2 :

$$\hat{a}_{2k} = \sum_{j=0}^{n/2-1} u_j (\omega^2)^{kj}, \quad \hat{a}_{2k+1} = \sum_{j=0}^{n/2-1} v_j (\omega^2)^{kj}.$$

Структура зеркальна DIT: у DIF естественный порядок входа и бит-реверсивный порядок выхода. Трудоёмкость также $\Theta(n \log n)$.

2.3 Общая шестиэтапная конструкция

Пусть $n = n'n''$. Запишем индекс входа как $j = n''u + v$, а индекс выхода как $k = n's + t$. Тогда

$$\omega^{kj} = \omega^{(n's+t)(n''u+v)} = (\omega^{n''})^{tu} \omega^{tv} (\omega^{n'})^{sv}.$$

Отсюда DFT раскладывается на:

1. представление входа как матрицы $n' \times n''$;
2. n'' независимых DFT размера n' по столбцам;
3. умножение на поворотные множители ω^{tv} ;
4. транспонирование;
5. n' независимых DFT размера n'' ;
6. финальную перестановку/считывание результата.

DIT получается при $n' = n/2, n'' = 2$, DIF — при $n' = 2, n'' = n/2$. При $n' \approx n'' \approx \sqrt{n}$ получается симметричная схема, удобная для иерархии памяти и параллельных вычислений.

На каждом уровне суммарный объём работы $\Theta(n)$, а сумма логарифмов размеров рекурсивных факторов равна $\log n$, поэтому общая сложность $\Theta(n \log n)$.

Каркас ответа на экзамене

Вывести две формулы бабочки DIT; записать $T(n) = 2T(n/2) + \Theta(n)$; показать формулы DIF; затем представить индекс как $j = n''u + v, k = n's + t$ и разложить показатель kj , что непосредственно даёт шесть этапов.

Билет 3. Вычислительные схемы и сети сортировки

Ключевая идея

Схема фиксирует все операции заранее. Её размер измеряет последовательную работу, глубина — идеальное параллельное время. Принцип нулей-единиц сводит корректность сети сравнения к бинарным входам.

3.1 Модель схем

Вычислительная схема — ориентированный ациклический граф с входами, выходами и вычислительными узлами. Последовательность операций не зависит от входа, то есть вычисление неадаптивно.

$$\text{size}(C) = \text{число узлов}, \quad \text{depth}(C) = \text{максимальная длина пути}.$$

Компаратор получает (x, y) и выдаёт $(\min(x, y), \max(x, y))$. Сеть слияния получает две отсортированные последовательности; сеть сортировки — произвольную последовательность.

3.2 Принцип нулей-единиц

Теорема 3.1. Сеть компараторов сортирует все входы из линейно упорядоченного множества тогда и только тогда, когда она сортирует все бинарные последовательности.

Доказательство. Необходимость очевидна. Предположим, что на некотором входе выход y не отсортирован: $y_k > y_l$ при $k < l$. Применим к каждому значению монотонное

отображение

$$\phi(x) = \begin{cases} 0, & x < y_k, \\ 1, & x \geq y_k. \end{cases}$$

Компаратор коммутирует с монотонным отображением:

$$\phi(\min(x, y)) = \min(\phi(x), \phi(y)), \quad \phi(\max(x, y)) = \max(\phi(x), \phi(y)).$$

Поэтому траектории элементов сохраняются, а в позициях k, l получаются 1, 0. Значит, сеть не сортирует некоторый бинарный вход. $\square$

3.3 Нечётно-чётное слияние Бэтчера

Для двух отсортированных последовательностей рекурсивно сливаются элементы нечётных позиций и отдельно элементы чётных позиций. Затем сравниваются соседние пары между двумя полученными подпоследовательностями.

Корректность удобно доказывать по принципу нулей-единиц. Каждая входная последовательность имеет вид $0^k 1^{n-k}$. После рекурсивных слияний числа нулей в двух выходах различаются не более чем на 2, поэтому после чередования возможна не более чем одна инверсия 10, которую исправляет заключительный слой компараторов.

Для слияния суммарной длины n :

$$S_M(n) = 2S_M(n/2) + \Theta(n) = \Theta(n \log n), \quad D_M(n) = D_M(n/2) + 1 = \Theta(\log n).$$

Сортировка двух половин и их слияние дают

$$S_{\text{sort}}(n) = 2S_{\text{sort}}(n/2) + \Theta(n \log n) = \Theta(n \log^2 n),$$

$$D_{\text{sort}}(n) = D_{\text{sort}}(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

3.4 Битонное слияние

Последовательность называется битонной, если она сначала монотонна в одном направлении, затем в другом; циклический сдвиг также допускается. Для $n = 2^r$ классическая схема сравнивает x_i с $x_{i+n/2}$, отправляя меньшие в первую половину, большие — во вторую. Обе половины остаются битонными, причём каждый элемент первой не больше каждого элемента второй; затем половины сливаются рекурсивно.

Эквивалентная форма из лекций сначала рекурсивно сливает нечётные и чётные позиции и затем сравнивает пары. Оценки:

$$S_{BM}(n) = \Theta(n \log n), \quad D_{BM}(n) = \Theta(\log n).$$

Битонная сортировка рекурсивно сортирует одну половину по возрастанию, другую по убыванию, после чего применяет битонное слияние:

$$S(n) = \Theta(n \log^2 n), \quad D(n) = \Theta(\log^2 n).$$

Каркас ответа на экзамене

Определить размер и глубину схемы, компаратор, сеть слияния и сортировки. Полностью доказать принцип нулей-единиц. Для двух сетей Бэтчера дать рекурсию, идею корректности и оценки размера/глубины.

Билет 4. Префиксное накопление, схема Ладнера–Фишера и BSP

Ключевая идея

Ассоциативность позволяет заменить линейную цепочку вычислений сбалансированным деревом. Схема Ладнера–Фишера вычисляет все префиксы с линейным размером и логарифмической глубиной.

4.1 Задача prefix scan

Для ассоциативной операции $\bullet$ и нейтрального элемента e требуется вычислить

$$b_i = a_0 \bullet a_1 \bullet \dots \bullet a_i, \quad 0 \leq i < n.$$

Последовательная цепочка имеет размер и глубину $n-1$. Сбалансированная схема уменьшает глубину до $\mathcal{O}(\log n)$.

4.2 Схема Ладнера–Фишера

Для простоты $n = 2^r$.

1. Параллельно вычислить произведения пар $p_j = a_{2j} \bullet a_{2j+1}$.
2. Рекурсивно вычислить префиксы $P_j = p_0 \bullet \dots \bullet p_j$.
3. Нечётные выходы равны $b_{2j+1} = P_j$; чётные восстанавливаются как $b_{2j} = P_{j-1} \bullet a_{2j}$, причём $b_0 = a_0$.

Для размера:

$$S(n) = S(n/2) + n/2 + (n/2 - 1) = S(n/2) + n - 1 = 2n - 2.$$

Для глубины достаточно двух слоёв на уровень:

$$D(n) = D(n/2) + 2 = 2 \log_2 n.$$

Существуют варианты с другим компромиссом “размер–глубина”, но указанные оценки уже оптимальны по порядку.

4.3 Модель BSP

BSP-компьютер задаётся параметрами:

- p процессоров с локальной памятью;
- g — стоимость передачи единицы данных (inverse bandwidth);
- l — стоимость барьерной синхронизации (latency).

Вычисление состоит из супершагов: локальная работа и коммуникация, затем барьер. Для супершага

$$\text{cost} = \text{comp} + g \text{ comm} + l,$$

где comp и comm берутся по наиболее нагруженному процессору. Для всего алгоритма:

$$\text{cost} = \text{comp} + g \text{ comm} + l \text{ sync}.$$

4.4 Ладнер–Фишер на BSP

Схему prefix можно рассматривать как восходящее дерево редукции и нисходящее дерево распространения префиксов. При допуске $n \geq p^2$ каждое дерево разбивается на верхний блок размера p и p нижних блоков размера n/p .

Один процессор вычисляет верхний блок, затем каждый процессор — свой нижний блок. Между фазами передаётся $\mathcal{O}(p)$ агрегированных значений; ввод и вывод распределены по процессорам. В принятой в лекциях модели получается

$$\text{comp} = \mathcal{O}(n/p), \quad \text{comm} = \mathcal{O}(n/p), \quad \text{sync} = \mathcal{O}(1).$$

Итоговая стоимость:

$$\mathcal{O}\left(\frac{n}{p} + g\frac{n}{p} + l\right).$$

Это оптимально по порядку по локальной работе и вводу-выводу.

Каркас ответа на экзамене

Сформулировать scan для произвольной ассоциативной операции; описать три шага рекурсии Ладнера–Фишера и вывести $2n - 2, 2 \log n$; определить p, g, l , comp, comm, sync и дать BSP-оценки.

Билет 5. Линейные рекурренты и параллельное сложение

Ключевая идея

Рекуррентное обновление первого порядка есть композиция аффинных преобразований. Композиция ассоциативна, следовательно, все состояния вычисляются одной задачей префиксного накопления.

5.1 Обобщённая линейная рекуррента

Дано

$$c_{-1} = 0, \quad c_i = a_i + b_i c_{i-1}.$$

Сопоставим шагу аффинное преобразование $f_i(x) = a_i + b_i x$. Композиция

$$(a, b) \star (c, d) = (a + bc, bd)$$

ассоциативна, поскольку соответствует композиции функций. Эквивалентно можно использовать матрицы

$$A_i = \begin{pmatrix} 1 & 0 \\ a_i & b_i \end{pmatrix}, \quad \begin{pmatrix} 1 \\ c_i \end{pmatrix} = A_i A_{i-1} \cdots A_0 \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Суффиксный scan матриц (или префиксный в обратном порядке) вычисляет все c_i схемой размера $\mathcal{O}(n)$ и глубины $\mathcal{O}(\log n)$.

Обобщение работает над полукольцом: вместо $+$ и $\cdot$ достаточно ассоциативных $\oplus, \odot$ и левой дистрибутивности

$$a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c).$$

Примеры: $(\min, +)$, $(\max, +)$, $(\vee, \wedge)$.

5.2 Сложение двоичных чисел

Пусть x_i, y_i — биты от младшего к старшему. Определим

$$g_i = x_i \wedge y_i \quad (\text{generate}), \quad p_i = x_i \text{ xor } y_i \quad (\text{propagate}).$$

Перенос удовлетворяет

$$c_{-1} = 0, \quad c_i = g_i \vee (p_i \wedge c_{i-1}).$$

Это линейная рекуррента над булевым полукольцом. После параллельного вычисления переносов

$$z_i = p_i \text{ xor } c_{i-1}, \quad z_n = c_{n-1}.$$

Итак, сумматор с ускоренным переносом имеет

$$\text{size} = \mathcal{O}(n), \quad \text{depth} = \mathcal{O}(\log n).$$

Для сравнения, ripple-carry сумматор имеет глубину $\Theta(n)$.

Каркас ответа на экзамене

Показать либо матричное представление, либо ассоциативную операцию над парами (a, b) . Затем подставить булевы $\vee, \wedge$, определить generate/propagate и получить логарифмическую глубину сложения.

Билет 6. Линейное программирование и кратчайшие пути

Ключевая идея

ЛР оптимизирует линейную функцию на многограннике. Основная теорема гарантирует оптимальную вершину. Кратчайшие расстояния характеризуются как максимально возможные потенциалы, удовлетворяющие всем треугольным ограничениям.

6.1 Формы ЛР

Допустимое решение удовлетворяет всем ограничениям; оптимальное допустимое решение достигает экстремума целевой функции.

Стандартная форма:

$$\max c^T x, \quad Ax \leq b, \quad x \geq 0.$$

Минимизацию заменяют максимизацией $-c^T x$, ограничение $a^T x \geq b$ умножают на -1 , равенство заменяют двумя неравенствами, свободную переменную представляют как $x = x^+ - x^-$.

Равновесная форма:

$$\max c^T x, \quad Ax = b, \quad x \geq 0.$$

Из стандартной формы она получается добавлением slack-переменных $s = b - Ax \geq 0$.

Теорема 6.1 (Основная теорема ЛР, без доказательства). Если задача ЛР разрешима и ограничена, то оптимальное решение существует. Если оптимум существует, то хотя бы одно оптимальное решение является вершиной допустимого многогранника.

6.2 Идея симплекс-метода

Симплекс-метод начинается с допустимой вершины и переходит по рёбрам многогранника в соседнюю вершину с лучшим значением целевой функции. Остановка означает локальную и одновременно глобальную оптимальность линейной функции. Метод эффективен практически, но имеет экспоненциальные худшие примеры. Полиномиальные альтернативы — метод эллипсоидов и методы внутренней точки.

6.3 Кратчайшие пути как LP

Пусть $G = (V, E)$ — ориентированный граф, $w : E \rightarrow \mathbb{R}$, источник s , все вершины достижимы и нет достижимого отрицательного цикла. Рассмотрим

$$\max \sum_{v \in V} x(v)$$

при ограничениях

$$x(s) = 0, \quad x(v) - x(u) \leq w(u, v) \quad ((u, v) \in E).$$

Лемма 6.2. Для любого допустимого x и любой вершины v выполнено $x(v) \leq \delta(s, v)$.

Доказательство. Для пути $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v$ сложим ограничения:

$$x(v_i) - x(v_{i-1}) \leq w(v_{i-1}, v_i).$$

Телескопическая сумма даёт $x(v) \leq w(P)$. Минимизируя по путям, получаем $x(v) \leq \delta(s, v)$. $\square$

Расстояния $x(v) = \delta(s, v)$ допустимы по неравенству треугольника

$$\delta(s, v) \leq \delta(s, u) + w(u, v).$$

Они покомпонентно доминируют любое допустимое решение, поэтому оптимальны для указанной целевой функции.

Если есть достижимый отрицательный цикл, система ограничений противоречива. Если есть недостижимая вершина, её потенциал можно увеличивать неограниченно, и LP неограничена.

Каркас ответа на экзамене

Дать обе формы LP и преобразования; сформулировать основную теорему; объяснить симплекс. Для кратчайших путей записать LP, доказать лемму суммированием вдоль пути и проверить допустимость δ .

Билет 7. Максимальный поток и алгоритм Форда–Фалкерсона

Ключевая идея

Остаточная сеть кодирует все допустимые локальные изменения потока. Отсутствие пути $s \rightsquigarrow t$ в ней одновременно даёт максимальный поток и минимальный разрез.

Пусть $G = (V, E)$, ёмкости $c_e \geq 0$, источник s , сток t . Поток f удовлетворяет

$$0 \leq f_e \leq c_e$$

и сохранению во всех $v \notin \{s, t\}$. Его значение $|f|$ — чистый выход из s (равно чистому входу в t).

LP максимального потока:

$$\max |f|, \quad \sum_{u:(u,v) \in E} f_{uv} - \sum_{w:(v,w) \in E} f_{vw} = 0, \quad 0 \leq f_e \leq c_e.$$

Для разреза $V = S \sqcup T$, $s \in S, t \in T$:

$$c(S, T) = \sum_{u \in S, v \in T} c_{uv}.$$

Всегда $|f| \leq c(S, T)$.

7.1 Остаточная сеть

Для прямой дуги остаточная ёмкость равна $c_{uv} - f_{uv}$, для обратной — f_{uv} . Путь $s \rightsquigarrow t$ в остаточной сети называется пополняющим. Его bottleneck

$$\Delta = \min_{e \in P} c_f(e) > 0$$

можно добавить по прямым дугам и вычесть по обратным.

Теорема 7.1 (Максимальный поток — минимальный разрез). *Следующие условия эквивалентны:*

- 1) f максимален;
- 2) в остаточной сети нет пополняющего пути;
- 3) существует разрез (S, T) , для которого $|f| = c(S, T)$.

Доказательство. Из 3) следует 1) по неравенству поток–разрез. Если есть пополняющий путь, значение потока можно увеличить, поэтому $1) \Rightarrow 2)$. Пусть пути нет. Возьмём S — множество вершин, достижимых из s в остаточной сети. Тогда $t \notin S$. Каждая исходная дуга $S \rightarrow T$ насыщена, иначе она была бы остаточной; каждая дуга $T \rightarrow S$ несёт нулевой поток, иначе существовала бы обратная остаточная дуга. Поэтому

$$|f| = \sum_{S \rightarrow T} f_e - \sum_{T \rightarrow S} f_e = \sum_{S \rightarrow T} c_e = c(S, T).$$

□

7.2 Форд–Фалкерсон

Алгоритм

Начать с $f = 0$. Пока в остаточной сети существует путь P из s в t , увеличить поток на bottleneck $\Delta(P)$. Если пути нет, вернуть f .

Корректность следует из теоремы. Если ёмкости целые, все остаточные ёмкости и приращения целые, поэтому каждая итерация увеличивает $|f|$ хотя бы на 1. При BFS/DFS за $\mathcal{O}(|E|)$:

$$T = \mathcal{O}(|E| |f^*|).$$

Это псевдополиномиальная оценка: $|f^*|$ может быть экспоненциальным относительно битовой длины ёмкостей. При иррациональных ёмкостях неудачный выбор путей может привести к незавершению.

Каркас ответа на экзамене

Определить поток, разрез и остаточную сеть; доказать тройную эквивалентность; описать Форд–Фалкерсона; для целых ёмкостей вывести число итераций $\leq |f^*|$ и время $\mathcal{O}(E|f^*|)$.

Билет 8. Двойственная LP, дополняющая нежесткость и слабая двойственность**Ключевая идея**

Двойственное решение задаёт линейную комбинацию ограничений, доминирующую целевую функцию. Поэтому любой двойственный допустимый вектор является верхней оценкой на любой первичный допустимый вектор.

Для стандартной первичной задачи

$$(P) \quad \max c^T x, \quad Ax \leq b, \quad x \geq 0$$

двойственная имеет вид

$$(D) \quad \min b^T y, \quad A^T y \geq c, \quad y \geq 0.$$

Переменная y_i соответствует i -му ограничению первичной задачи, а j -е двойственное ограничение — переменной x_j .

Для равновесной формы:

$$\max c^T x, Ax = b, x \geq 0 \iff \min b^T y, A^T y \geq c,$$

где y свободен по знаку. Свободная первичная переменная, наоборот, порождает двойственное равенство.

8.1 Дополняющая нежесткость

Для допустимых x, y условия имеют вид

$$y_i(b_i - (Ax)_i) = 0 \quad \forall i,$$

$$x_j((A^T y)_j - c_j) = 0 \quad \forall j.$$

То есть положительной переменной соответствует жесткое противоположное ограничение.

Лемма 8.1. Если допустимые x, y удовлетворяют условиям дополняющей нежесткости, то $c^T x = b^T y$.

Доказательство. Из второй группы условий

$$c^T x = (A^T y)^T x = y^T Ax.$$

Из первой группы $y^T Ax = y^T b = b^T y$. □

Теорема 8.2 (Слабая двойственность). Для любых допустимых x и y выполнено

$$c^T x \leq b^T y.$$

Доказательство. Поскольку $A^T y \geq c$ и $x \geq 0$,

$$c^T x \leq y^T A x.$$

Поскольку $Ax \leq b$ и $y \geq 0$,

$$y^T A x \leq y^T b = b^T y.$$

□

Если для допустимой пары достигнуто равенство, оба решения оптимальны: между ними не может поместиться лучшее значение.

Каркас ответа на экзамене

Из стандартной формы механически построить двойственную; объяснить правила для равенств и свободных переменных; записать две группы complementary slackness; доказать лемму и слабую двойственность одной цепочкой неравенств.

Билет 9. Сильная теорема двойственности

Ключевая идея

При существовании допустимых решений оптимальные значения первичной и двойственной задач совпадают. Геометрически градиент целевой функции в оптимальной вершине является неотрицательной комбинацией нормалей активных ограничений.

Теорема 9.1 (Сильная двойственность). Пусть первичная и двойственная задачи имеют допустимые решения. Для допустимых x, y эквивалентны:

- 1) x, y оптимальны;
- 2) $c^T x = b^T y$;
- 3) выполняются условия дополняющей нежесткости.

В частности, оптимальные значения совпадают.

Импlications 3) $\Rightarrow$ 2) и 2) $\Rightarrow$ 1) уже следуют из предыдущего билета. Содержательная часть — существование двойственного оптимума, удовлетворяющего complementary slackness.

9.1 Геометрическое доказательство в невырожденном случае

Рассмотрим форму без ограничений знака у x :

$$(P) \quad \max c^T x, \quad Ax \leq b,$$

$$(D) \quad \min b^T y, \quad A^T y = c, \quad y \geq 0.$$

Пусть оптимальная вершина x^* определяется ровно n линейно независимыми активными ограничениями. После перенумерации

$$A_{\bullet} x^* = b_{\bullet}, \quad A_{\bullet} \in \mathbb{R}^{n \times n} \text{ невырождена.}$$

Определим

$$y_{\bullet}^* = (A_{\bullet}^{-1})^T c, \quad y_i^* = 0 \text{ для неактивных ограничений.}$$

Тогда $A^T y^* = c$, и complementary slackness выполнена по построению.

Остаётся доказать $y_j^* \geq 0$. Для j рассмотрим малое движение

$$x^{(j)} = x^* - \varepsilon A_{\bullet}^{-1} e_j.$$

Оно ослабляет j -е активное ограничение и при достаточно малом ε не нарушает неактивные. Оптимальность x^* даёт

$$0 \geq c^T (x^{(j)} - x^*) = -\varepsilon c^T A_{\bullet}^{-1} e_j = -\varepsilon y_j^*,$$

следовательно, $y_j^* \geq 0$. Значит, y^* допустимо. Далее

$$b^T y^* = b_{\bullet}^T (A_{\bullet}^{-1})^T c = (A_{\bullet}^{-1} b_{\bullet})^T c = (x^*)^T c.$$

По слабой двойственности y^* оптимально.

В вырожденном случае выбирают линейно независимую подсистему активных ограничений после стандартного возмущения или используют лемму Фаркаша; результат остаётся тем же.

Наконец, если y^{**} — любое двойственное оптимальное решение, то

$$(Ax^* - b)^T y^{**} = x^{*T} A^T y^{**} - b^T y^{**} = c^T x^* - b^T y^{**} = 0,$$

то есть complementary slackness переносится на любую оптимальную пару.

Каркас ответа на экзамене

Сформулировать эквивалентность трёх условий. Две лёгкие импликации сослать на слабую двойственность. Для трудной части взять активную матрицу $A_{\bullet}$, построить $y^* = (A_{\bullet}^{-1})^T c$, доказать $y^* \geq 0$ малым движением и получить равенство целей.

Билет 10. Дробный минимальный разрез как двойственная задача

Ключевая идея

Двойственные переменные максимального потока образуют потенциал вершин и плату за дуги, пересекающие падение потенциала. Случайный порог превращает любой дробный потенциал в обычный разрез без увеличения ожидаемой стоимости.

После двойствования LP максимального потока и нормировки $z(s) = 1, z(t) = 0$ получается

$$\min \sum_{(u,v) \in E} c_{uv} y_{uv}$$

при

$$y_{uv} \geq z(u) - z(v), \quad y_{uv} \geq 0.$$

Можно ограничиться $0 \leq z(v) \leq 1$. Для фиксированного z оптимальный выбор

$$y_{uv} = \max\{z(u) - z(v), 0\}.$$

Это дробный (s, t) -разрез.

Теорема 10.1 (Целочисленность вероятностным методом). У дробного минимального разреза существует оптимальное решение c $z(v), y_{uv} \in \{0, 1\}$.

Доказательство. Пусть (y, z) — допустимое дробное решение. Выберем $\tau \sim \text{Unif}[0, 1]$ и положим

$$S_\tau = \{v : z(v) \geq \tau\}, \quad T_\tau = V \setminus S_\tau.$$

Так как $z(s) = 1, z(t) = 0$, это (s, t) -разрез. Для дуги (u, v)

$$\mathbb{P}(u \in S_\tau, v \in T_\tau) = \max\{z(u) - z(v), 0\} = y_{uv}.$$

По линейности ожидания

$$\mathbb{E}c(S_\tau, T_\tau) = \sum_{uv} c_{uv} \mathbb{P}(u \in S_\tau, v \in T_\tau) = \sum_{uv} c_{uv} y_{uv}.$$

Следовательно, существует конкретный порог, для которого стоимость разреза не больше дробной цели. Если дробное решение оптимально, полученный целочисленный разрез также оптимален. $\square$

В сочетании с сильной двойственностью это снова даёт равенство максимального потока и минимального разреза.

Каркас ответа на экзамене

Записать $z(s) = 1, z(t) = 0, y_{uv} \geq z(u) - z(v), 0$; минимизировать y при фиксированном z ; выбрать равномерный порог и посчитать вероятность пересечения каждой дуги; применить линейность ожидания.

Билет 11. Приближённые алгоритмы и вершинное покрытие

Ключевая идея

Для минимизационной задачи ρ -приближение имеет стоимость не больше ρOPT . Для vertex cover коэффициент 2 получается либо из максимального паросочетания, либо округлением LP по порогу $1/2$.

11.1 Определение

Для минимизации алгоритм является ρ -приближённым, если для любого входа

$$\text{cost}(A) \leq \rho \text{OPT}, \quad \rho \geq 1.$$

Для максимизации требуется $\text{value}(A) \geq \text{OPT}/\rho$.

11.2 Невзвешенное вершинное покрытие

Вершинное покрытие $C \subseteq V$ содержит хотя бы один конец каждого ребра.

Алгоритм

Построить максимальное по включению паросочетание M жадно: брать произвольное оставшееся ребро и удалять все инцидентные ему рёбра. Вернуть оба конца всех рёбер M .

Возвращённое множество покрывает граф: иначе непокрытое ребро можно было бы добавить к M . Любое покрытие содержит хотя бы один конец каждого ребра паросочетания, поэтому

$$\text{OPT} \geq |M|, \quad |C| = 2|M| \leq 2\text{OPT}.$$

Время при списках смежности $\mathcal{O}(|E|)$.

11.3 Взвешенное покрытие через LP

Пусть стоимость вершины $c_v \geq 0$. LP-релаксация:

$$\min \sum_v c_v x_v, \quad x_u + x_v \geq 1 \ (uv \in E), \quad x_v \geq 0.$$

Пусть x^* оптимально. Округлим

$$\bar{x}_v = \mathbf{1}[x_v^* \geq 1/2].$$

Для каждого ребра хотя бы один конец имеет $x^* \geq 1/2$, поэтому $\bar{x}$ — покрытие. Кроме того,

$$\sum_v c_v \bar{x}_v \leq \sum_v 2c_v x_v^* = 2 \text{LP}^* \leq 2 \text{OPT}.$$

Решение LP и округление дают полиномиальный 2-приближённый алгоритм.

Каркас ответа на экзамене

Определить коэффициент приближения. Для невзвешенного случая: максимальное паросочетание, покрытие из концов, нижняя оценка $|M|$. Для взвешенного: LP, порог $1/2$, две строки доказательства допустимости и стоимости.

Билет 12. Метод двойственной допустимости для weighted vertex cover

Ключевая идея

Вместо решения LP полностью можно постепенно строить допустимое двойственное решение. Когда ограничение вершины становится жёстким, вершина включается в покрытие. Каждая двойственная переменная оплачивается не более двух раз.

Прямая релаксация:

$$\min \sum_v c_v x_v, \quad x_u + x_v \geq 1, \quad x_v \geq 0.$$

Двойственная:

$$\max \sum_{e \in E} y_e, \quad \sum_{e \ni v} y_e \leq c_v, \quad y_e \geq 0.$$

Определим slack вершины

$$\Delta(v) = c_v - \sum_{e \ni v} y_e.$$

Алгоритм

Начать с $C = \emptyset$, $y = 0$. Пока есть непокрытое ребро $e = uv$, увеличить y_e на

$$\delta = \min\{\Delta(u), \Delta(v)\}.$$

Добавить в C каждый конец, для которого slack стал нулём. Продолжать, пока все рёбра не покрыты.

Двойственная допустимость сохраняется. Каждая выбранная вершина жёсткая:

$$c_v = \sum_{e \ni v} y_e.$$

Поэтому

$$\begin{aligned} c(C) &= \sum_{v \in C} c_v = \sum_{v \in C} \sum_{e \ni v} y_e \\ &\leq 2 \sum_{e \in E} y_e \leq 2\text{OPT}, \end{aligned}$$

где последнее неравенство — слабая двойственность. Алгоритм можно реализовать за $O(|E|)$ при аккуратном хранении slack.

Метод называется primal-dual: двойственное решение всё время допустимо и улучшается; частичное целочисленное первичное решение поддерживает приближённую форму complementary slackness.

Каркас ответа на экзамене

Выписать прямую и двойственную LP; определить slack; описать подъём y_e до жёсткости конца; доказать $c(C) \leq 2 \sum y_e \leq 2\text{OPT}$.

Билет 13. Метрическая TSP: двойное дерево и Кристофидес–Сердюков

Ключевая идея

MST является нижней оценкой на оптимальный тур. Двойное дерево делает все степени чётными ценой ещё одного MST; Кристофидес исправляет только нечётные степени минимальным паросочетанием.

В метрической TSP граф полный, веса неотрицательны и удовлетворяют

$$w(x, z) \leq w(x, y) + w(y, z).$$

Спрявление повторной вершины в обходе не увеличивает стоимость.

Пусть C^* — оптимальный гамильтонов цикл, T — MST. Удаление одного ребра из C^* даёт остовное дерево, поэтому

$$w(T) \leq w(C^*).$$

13.1 Алгоритм двойного дерева

Удвоить все рёбра T , найти эйлеров цикл и спрямить повторные вершины. Стоимость до спрямления равна $2w(T)$, следовательно

$$w(C) \leq 2w(T) \leq 2w(C^*).$$

13.2 Алгоритм Кристофидеса–Сердюкова

Пусть O — множество вершин нечётной степени в T ; $|O|$ чётно. Найти минимальное совершенное паросочетание M в полном подграфе на O . Граф $T \cup M$ связан и имеет только чётные степени, поэтому эйлеров. После спрямления получаем тур.

Лемма 13.1. $w(M) \leq \frac{1}{2}w(C^*)$.

Доказательство. Расположим вершины O в порядке их появления на оптимальном цикле. Участки C^* между соседними вершинами O образуют цикл на O после метрического спрямления, стоимость которого не больше $w(C^*)$. Чередующиеся рёбра этого цикла дают два совершенных паросочетания, сумма их стоимостей не больше $w(C^*)$. Более дешёвое имеет стоимость не больше половины; минимальное M не дороже. $\square$

Следовательно,

$$w(C) \leq w(T) + w(M) \leq w(C^*) + \frac{1}{2}w(C^*) = \frac{3}{2}w(C^*).$$

Все этапы полиномиальны: MST, минимальное совершенное паросочетание, эйлеров цикл и спрямление.

Каркас ответа на экзамене

Доказать нижнюю оценку MST. Для double-tree: удвоение, Эйлер, shortcut, коэффициент 2. Для Кристофидеса: нечётные вершины, минимальное matching, лемма $w(M) \leq OPT/2$, итог $3/2$.

Билет 14. Алгоритм Фрейвальдса

Ключевая идея

Вместо сравнения матриц AB и C достаточно сравнить их действие на случайный бинарный вектор. Неверная строка обнаруживается с вероятностью не меньше $1/2$.

Для $A, B, C \in R^{n \times n}$:

Алгоритм

Выбрать $r \in \{0, 1\}^n$ равномерно. Вычислить $u = Br$, $v = Au$, $w = Cr$. Принять тогда и только тогда, когда $v = w$.

Лемма 14.1. Если $x \neq y \in R^n$ и $r \in \{0, 1\}^n$ равномерен, то

$$\mathbb{P}(x^T r = y^T r) \leq \frac{1}{2}.$$

Доказательство. Выберем j с $x_j \neq y_j$ и зафиксируем все r_i , $i \neq j$. Из двух значений $r_j \in \{0, 1\}$ не более одного может дать равенство, поскольку разность двух уравнений равна $x_j - y_j \neq 0$. $\square$

Если $AB = C$, алгоритм всегда принимает. Если $AB \neq C$, есть различная строка, и лемма даёт вероятность обнаружения хотя бы $1/2$. Три умножения матрицы на вектор требуют $\mathcal{O}(n^2)$. После k независимых повторов ошибка не больше 2^{-k} , время $\mathcal{O}(kn^2)$.

Каркас ответа на экзамене

Алгоритм $A(Br)$ против Cr ; лемма с фиксацией всех битов, кроме одного; односторонняя ошибка; усиление повторением.

Билет 15. Проверка равенства многочленов и лемма Шварца–Зиппеля

Ключевая идея

Ненулевой многочлен малой степени не может обращаться в ноль на слишком большой доле декартова произведения конечного множества.

Теорема 15.1 (Шварц–Зиппель). Пусть $p \in \mathbb{F}[x_1, \dots, x_n]$ ненулевой, его полная степень равна d , $S \subseteq \mathbb{F}$ конечно, а r_i независимы и равномерны в S . Тогда

$$\mathbb{P}(p(r_1, \dots, r_n) = 0) \leq \frac{d}{|S|}.$$

Доказательство. Индукция по n . Для $n = 1$ ненулевой многочлен степени d имеет не более d корней. Для перехода запишем

$$p(x_1, \dots, x_n) = \sum_{i=0}^k x_1^i q_i(x_2, \dots, x_n), \quad q_k \neq 0.$$

Имеем $\deg q_k \leq d - k$. По индукции вероятность $q_k(r_2, \dots, r_n) = 0$ не больше $(d - k)/|S|$. Если старший коэффициент не нулевой, получившийся одночленный многочлен по x_1 имеет не более k корней, то есть условная вероятность не больше $k/|S|$. Сложение оценок даёт $d/|S|$. $\square$

Для проверки $p \equiv q$ применяют лемму к $p - q$: выбирают случайную точку и сравнивают значения. Ошибка односторонняя: тождественно равные многочлены всегда принимаются; для разных вероятность ложного принятия не больше $d/|S|$. После k повторов:

$$\mathbb{P}(\text{ошибка}) \leq (d/|S|)^k.$$

Если многочлен задан арифметической схемой и вычисляется за T_p , время равно $\mathcal{O}(kT_p)$ без раскрытия экспоненциального числа коэффициентов.

Типичная ошибка

Над малым конечным полем равенство как функций может не означать равенства формальных многочленов: например, $x^2 - x$ задаёт нулевую функцию на $\mathbb{F}_2$, но не является нулевым многочленом.

Каркас ответа на экзамене

Сформулировать лемму с полной степенью; доказать индукцией через старший коэффициент по x_1 ; описать РИТ и усиление вероятности.

Билет 16. Универсальное хеширование и семейство multiply-add

Ключевая идея

Случайная функция выбирается после того, как ключи определены. Универсальность гарантирует для каждой фиксированной пары вероятность коллизии не больше $1/m$.

Пусть U — пространство ключей, $M = \{0, \dots, m-1\}$. Семейство $\mathcal{H} \subseteq M^U$ называется ε -универсальным, если для любых $x \neq y$

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] \leq \varepsilon.$$

При $\varepsilon = 1/m$ говорят просто “универсальное”.

Пусть $U \subseteq \mathbb{F}_p$, $p > |U|$, $a \neq 0$, $a, b \in \mathbb{F}_p$:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Теорема 16.1. Семейство *multiply-add* универсально.

Доказательство. Для фиксированных $x \neq y$ отображение

$$(a, b) \mapsto (r, s) = (ax + b, ay + b)$$

является биекцией между парами $a \neq 0$ и упорядоченными парами $r \neq s$: обратное решение линейной системы единственно,

$$a = (r - s)(x - y)^{-1}, \quad b = (sx - ry)(x - y)^{-1}.$$

Значит, r, s равномерны среди различных элементов $\mathbb{F}_p$. Для каждого r не более $\lfloor p/m \rfloor \leq (p-1)/m$ значений $s \neq r$ имеют тот же остаток modulo m . Поэтому вероятность коллизии не больше $1/m$. $\square$

Параметры a, b хранятся в $\mathcal{O}(1)$ машинных словах, однако произведение ax должно помещаться в доступный расширенный тип; это практическое ограничение метода.

Каркас ответа на экзамене

Определить универсальность в худшем случае по паре ключей. Для *multiply-add* доказать биекцию $(a, b) \leftrightarrow (r, s)$ и затем посчитать пары с одинаковым остатком modulo m .

Билет 17. Семейство *multiply-shift*

Ключевая идея

Умножение на случайное нечётное слово перемешивает биты; хешом берутся старшие ℓ битов произведения modulo 2^w . Вероятность коллизии не превосходит $2/m$.

Пусть $U = \mathbb{Z}_{2^w}$, $m = 2^\ell$, а a выбирается равномерно среди нечётных w -битных чисел. Определим

$$h_a(x) = \left\lfloor \frac{(ax \bmod 2^w)}{2^{w-\ell}} \right\rfloor,$$

то есть берём окно старших ℓ битов.

Теорема 17.1. Семейство $\{h_a : a \text{ нечётно}\}$ является $(2/m)$ -универсальным.

Схема доказательства. Пусть $d = y - x = q2^r$, где q нечётно. Умножение q на случайное нечётное a переставляет нечётные элементы modulo 2^w , поэтому $z = aq$ равномерен среди нечётных слов. Тогда

$$ad = z2^r \pmod{2^w}$$

имеет фиксированные младшие $r+1$ битов $0 \dots 01$, а старшие биты равномерны.

Если $h_a(x) = h_a(y)$, то при вычитании $ay - ax$ окно старших битов разности может быть только 0 либо $m - 1$ (в зависимости от займа через границу слова). Для каждого из этих двух значений вероятность не больше $1/m$; в пограничном случае одна вероятность равна $2/m$, а другая нулю. По неравенству Буля

$$\mathbb{P}(h_a(x) = h_a(y)) \leq \frac{2}{m}.$$

□

Метод особенно эффективен на реальном процессоре: переполнение беззнакового слова автоматически вычисляет modulo 2^w , затем остаётся сдвиг вправо.

Каркас ответа на экзамене

Дать формулу через старшие биты; разложить $y - x = q2^r$; объяснить, почему aq равномерен среди нечётных; коллизия сводится к двум опасным значениям окна; итог $2/m$.

Билет 18. Совершенное и двухуровневое хеширование

Ключевая идея

Для статического множества можно подобрать функции без коллизий. Квадратичная таблица делает это сразу; двухуровневая схема FKS сохраняет отсутствие коллизий при линейной суммарной памяти.

Пусть фиксировано множество $S \subset U$, $|S| = s$. Совершенная хеш-функция инъективна на S и обеспечивает поиск за $\mathcal{O}(1)$ в худшем случае.

18.1 Один уровень

Возьмём универсальное семейство в таблицу размера $m = s^2$. По union bound:

$$\mathbb{P}(h|_S \text{ неинъективна}) \leq \binom{s}{2} \frac{1}{s^2} < \frac{1}{2}.$$

Поэтому случайный выбор находит инъективную функцию в среднем менее чем за две попытки.

18.2 Два уровня FKS

Первичная таблица имеет $m = s$ ячеек. Пусть S_i — ключи в bucket i , $s_i = |S_i|$. Для bucket строится вторичная таблица размера $m_i = s_i^2$ и подбирается функция без коллизий.

Ключевая оценка:

$$\mathbb{E} \sum_i s_i^2 = s + 2\mathbb{E} \sum_i \binom{s_i}{2} \leq s + 2 \binom{s}{2} \frac{1}{s} < 2s.$$

По Маркову

$$\mathbb{P} \left(\sum_i s_i^2 > 4s \right) < \frac{1}{2}.$$

Перебирая первичную функцию, получаем суммарный вторичный размер $\mathcal{O}(s)$ в среднем за константное число попыток. В каждом bucket вторичная функция находится также за ожидаемое константное число попыток.

Поиск выполняет ровно два вычисления хеша и одно обращение к памяти:

$$T_{\text{query}} = \mathcal{O}(1) \quad \text{в худшем случае,} \quad \text{память } \mathcal{O}(s).$$

Схема статическая: при изменении множества обычно требуется перестроение.

Каркас ответа на экзамене

Сначала доказать вероятность успеха для $m = s^2$. Затем FKS: bucket sizes s_i , вторичные размеры s_i^2 , вычислить $\mathbb{E} \sum s_i^2 < 2s$, применить Маркова, завершить $\mathcal{O}(s)$ памятью и worst-case $\mathcal{O}(1)$ поиском.

Билет 19. Разрешение коллизий: списки и линейное зондирование

Ключевая идея

При списках универсальность контролирует ожидаемую длину bucket. При линейном зондировании основная опасность — длинные кластеры; при малой загрузке вероятность длинного заполненного блока экспоненциально убывает.

Пусть хранится n ключей в таблице размера m , $\alpha = n/m$.

19.1 Хеширование списками

В ячейке i хранится список ключей с $h(x) = i$. Для фиксированного запроса x и универсального случайного h :

$$\mathbb{E} \ell(h(x)) \leq 1 + \sum_{y \neq x} \mathbb{P}(h(y) = h(x)) \leq 1 + \frac{n-1}{m} < 1 + \alpha.$$

Отсюда ожидаемое время поиска $\mathcal{O}(1 + \alpha)$, вставка в начало списка — $\mathcal{O}(1)$.

19.2 Линейное зондирование

Пробная последовательность:

$$h(x), h(x) + 1, h(x) + 2, \dots \pmod{m}.$$

Максимальный заполненный циклический отрезок называется кластером. Стоимость операции пропорциональна длине кластера, в который попал $h(x)$.

В упрощённом анализе предполагается полностью случайная функция h и $\alpha \leq 1/2$. Для фиксированного отрезка длины r вероятность того, что в него хешируется не меньше r ключей, оценивается биномиально:

$$\mathbb{P}(X \geq r) \leq \binom{n}{r} \left(\frac{r}{m}\right)^r \leq (e\alpha)^r.$$

Более аккуратный учёт максимальной длины блока даёт основание меньше 1, например $\sqrt{e/2} < 1$ в оценке лекций. Поскольку заданный хеш может лежать в r позициях блока и просмотреть $\mathcal{O}(r)$ ячеек,

$$\mathbb{E} T(x) \leq \sum_{r \geq 1} \mathcal{O}(r^2) q^r = \mathcal{O}(1), \quad q < 1.$$

Это не анализ универсального хеширования: линейное зондирование требует более сильной случайности из-за кластеризации.

Типичная ошибка

Удаление нельзя реализовать простым обнулением ячейки: это разрывает пробную цепочку. Используют tombstone либо специальное обратное сдвигание элементов.

Каркас ответа на экзамене

Для chaining вывести $1 + \alpha$ суммой индикаторов. Для linear probing определить кластер, оценить вероятность блока длины r экспонентой и просуммировать $\sum r^2 q^r$.

Билет 20. Бинарное зондирование

Ключевая идея

XOR-пробирование просматривает ячейки по иерархии двоичных отрезков. Долгий поиск возможен только при существовании “востребованного” плотного dyadic-отрезка, вероятность которого быстро убывает.

Пусть $m = 2^\ell$. Последовательность проб:

$$h(x), h(x) \text{ xor } 1, h(x) \text{ xor } 2, h(x) \text{ xor } 3, \dots$$

Первые 2^k проб заполняют в некотором порядке стандартный двоичный отрезок $B_k(x)$ длины 2^k , содержащий $h(x)$. Исключая повторения, алгоритм последовательно просматривает sibling-отрезки на возрастающих уровнях.

Назовём dyadic-отрезок B *востребованным*, если число ключей, чьи исходные хеши лежат в B , не меньше $|B|$. Востребованный отрезок обязательно заполнен. Обратное не всегда верно, но структурный аргумент показывает:

Лемма 20.1. Если некоторый предок $B_k(x)$ уже заполнен, то либо он сам, либо один из его “авункуляров” (соседних sibling-отрезков предков) востребован.

При полностью случайном h , $\alpha \leq 1/4$, для фиксированного отрезка длины $r = 2^k$:

$$q_k = \mathbb{P}(B \text{ востребован}) \leq \binom{n}{r} (r/m)^r \leq (e\alpha)^r.$$

Для достаточно больших k это не больше 4^{-k} с большим запасом, так как фактически убывает как c^{2^k} . Вероятность заполненности предка оценивается суммой вероятностей востребованности его самого и авункуляров:

$$p_k = \mathcal{O}(4^{-k}).$$

Стоимость достижения уровня k равна $\mathcal{O}(2^k)$, поэтому

$$\mathbb{E}T(x) \leq \sum_{k=0}^{\ell-1} \mathcal{O}(2^k) p_k = \sum_{k \geq 0} \mathcal{O}(2^{-k}) = \mathcal{O}(1).$$

Каркас ответа на экзамене

Объяснить XOR-порядок через dyadic-дерево, определить востребованный отрезок, сформулировать структурную лемму, оценить q_k биномиально и просуммировать $2^k p_k$.

Билет 21. SAT, k -SAT и детерминированный алгоритм для 2-SAT**Ключевая идея**

Каждый дизъюнкт 2-CNF эквивалентен двум импликациям. Формула невыполнима ровно тогда, когда переменная и её отрицание взаимно достижимы, то есть лежат в одной SCC.

SAT спрашивает, существует ли набор $x \in \{0, 1\}^n$, на котором булева формула истинна. В k -SAT формула задана в КНФ, и каждый дизъюнкт содержит не более (или ровно, в конвенции лекций) k литералов.

Для 2-SAT используем

$$(P \vee Q) \equiv (\neg P \Rightarrow Q) \wedge (\neg Q \Rightarrow P).$$

Строим граф импликаций с вершинами $x_i, \neg x_i$. Для каждого clause $(L \vee M)$ добавляем дуги

$$\neg L \rightarrow M, \quad \neg M \rightarrow L.$$

Теорема 21.1. 2-CNF выполнима тогда и только тогда, когда ни для одного i литералы x_i и $\neg x_i$ не лежат в одной компоненте сильной связности.

Доказательство. Если они в одной SCC, то существуют импликации $x_i \Rightarrow \neg x_i$ и $\neg x_i \Rightarrow x_i$. При любом значении один из этих истинных литералов вынуждает ложный, противоречие.

Обратно, сожмём SCC в DAG. Компонента отрицаний $\neg C$ определена корректно, и $C \neq \neg C$. Рассматриваем компоненты в обратном топологическом порядке. Если переменная ещё не назначена, присваиваем всем литералам текущей компоненты значение true, а литералам в противоположной компоненте — false. Дуга из истинной компоненты не может вести в уже ложную: это противоречило бы порядку обработки. Следовательно, все импликации, а значит и все clauses, выполнены. $\square$

SCC вычисляются алгоритмом Косарайю, Тарьяна или Габова за

$$\mathcal{O}(|V| + |E|) = \mathcal{O}(n + m).$$

Каркас ответа на экзамене

Дать определения SAT и k -SAT; заменить clause двумя импликациями; построить граф; доказать SCC-критерий в обе стороны; указать линейное время.

Билет 22. Вероятностный алгоритм для 2-SAT

Ключевая идея

При фиксированном удовлетворяющем наборе изменение случайной переменной ложного clause не увеличивает расстояние Хэмминга в среднем. Получается симметричное случайное блуждание с ожидаемым временем достижения нуля не больше n^2 .

Алгоритм

Начать с произвольного набора x . Пока формула ложна, выбрать любой ложный clause $(L_i \vee L_j)$ и равновероятно переключить одну из двух его переменных. Выполнить не более $2n^2$ шагов; если решение не найдено, ответить “невыполнима”.

Пусть x^* — фиксированное решение, $D_t = d_H(x_t, x^*)$. В ложном clause хотя бы один из двух текущих битов отличается от соответствующего бита x^* .

- если отличаются оба, следующий шаг уменьшает D_t на 1;
- если отличается один, D_t увеличивается или уменьшается на 1 с вероятностями $1/2$.

Значит, процесс не хуже симметричного блуждания на $\{0, \dots, n\}$ с поглощением в 0 и отражением в n .

Пусть E_i — ожидаемое время достижения 0 из расстояния i :

$$E_0 = 0, \quad E_i = 1 + \frac{E_{i-1} + E_{i+1}}{2} \quad (0 < i < n), \quad E_n = 1 + E_{n-1}.$$

Решение:

$$E_i = 2in - i^2 \leq n^2.$$

По неравенству Маркова вероятность не найти решение за $2n^2$ шагов не больше $1/2$. Ошибка односторонняя: найденный набор всегда проверяется; невыполнимая формула никогда не объявляется выполнимой. При наивном поиске ложного clause время $\mathcal{O}(mn^2)$.

Каркас ответа на экзамене

Описать random walk; зафиксировать x^* ; разобрать два случая для ложного clause; выписать систему для E_i и решение $2in - i^2$; применить Маркова.

Билет 23. Вероятностный алгоритм для 3-SAT

Ключевая идея

Алгоритм Шёнинга многократно запускает короткое случайное блуждание из случайной точки. Одна попытка успешна с вероятностью порядка $(3/4)^n$ с точностью до полиномиального множителя.

Алгоритм

1. Выбрать $x \in \{0, 1\}^n$ равномерно.
2. Не более $3n$ раз: если формула выполнена, вернуть x ; иначе выбрать ложный 3-clause и случайно переключить одну из трёх его переменных.
3. Перезапустить алгоритм независимо.

Пусть x^* — решение. В ложном clause хотя бы одна переменная отличается от x^* , поэтому вероятность уменьшить расстояние Хэмминга хотя бы $1/3$. Стандартный анализ считает траектории, в которых из начального расстояния i блуждание делает достаточно исправляющих шагов. С учётом биномиального распределения начального расстояния и оценки Стирлинга:

$$\mathbb{P}(\text{успех одной попытки}) = \Omega\left(\frac{1}{\sqrt{n}} \left(\frac{3}{4}\right)^n\right).$$

Следовательно, после

$$R = \Theta\left(\sqrt{n} \left(\frac{4}{3}\right)^n\right)$$

независимых попыток вероятность неудачи ограничена константой, например

$$(1 - p)^R \leq e^{-pR} \leq e^{-1}.$$

Общее время

$$\mathcal{O}^*((4/3)^n),$$

где $\mathcal{O}^*$ скрывает полиномиальные множители. Ошибка односторонняя.

Каркас ответа на экзамене

Дать одну попытку Шёнинга (случайный старт, $3n$ шагов); объяснить вероятность движения к фиксированному решению; сформулировать лемму об успехе $\Omega(n^{-1/2}(3/4)^n)$; выбрать число перезапусков и получить $\mathcal{O}^*((4/3)^n)$.

Билет 24. Алгоритм Миллера–Рабина

Ключевая идея

Для простого нечётного p цепочка последовательных квадратов от a^d к a^{p-1} может прийти к 1 только через -1 . Нарушение этого свойства является сертификатом составности.

Пусть проверяемое нечётное $N > 2$ и

$$N - 1 = 2^s d, \quad d \text{ нечётно.}$$

Алгоритм

Выбрать $a \in \{2, \dots, N - 2\}$. Вычислить $x = a^d \bmod N$. Если $x \in \{1, N - 1\}$, раунд пройден. Иначе повторить до $s - 1$ раз: $x \leftarrow x^2 \bmod N$; если $x = N - 1$, раунд пройден; если $x = 1$ раньше появления -1 , вернуть “составное”. Если -1 не появился, вернуть “составное”; иначе “вероятно простое”.

Если N простое, малая теорема Ферма даёт $a^{N-1} = 1$, а уравнение $z^2 = 1$ в поле имеет только корни ± 1 . Поэтому алгоритм никогда не отвергает простое.

Для составного N основания, на которых тест принимает, образуют малую часть мультипликативной группы. Достаточная для курса оценка:

$$\mathbb{P}(\text{ложное принятие за один раунд}) \leq \frac{1}{2}.$$

Классическая усиленная теорема даёт не более $1/4$ для любого нечётного составного N . После k независимых раундов ошибка не больше 2^{-k} (или 4^{-k} при усиленной оценке).

Быстрое возведение в степень использует $\mathcal{O}(\log N)$ модульных умножений. При школьном умножении $n = \lceil \log_2 N \rceil$ -битных чисел:

$$T = \mathcal{O}(n^3)$$

на раунд; с быстрым умножением — $\mathcal{O}(M(n) \log n)$.

Каркас ответа на экзамене

Разложить $N - 1 = 2^s d$; выписать цепочку $a^d, a^{2d}, \dots, a^{2^s d}$; объяснить “нет нетривиального квадратного корня из 1” для простого; дать одностороннюю ошибку и усиление повторением.

Билет 25. Онлайн-алгоритмы: аренда и кэширование

Ключевая идея

Онлайн-алгоритм принимает решения без знания будущего и сравнивается с оптимальным ясновидящим алгоритмом. Для аренды оптимален порог покупки; для кэширования случайная маркировка даёт логарифмическое конкурентное отношение.

Для минимизационной задачи конкурентное отношение

$$\alpha_A = \sup_{\sigma} \frac{\text{cost}_A(\sigma)}{\text{OPT}(\sigma)}.$$

Иногда допускают аддитивную константу; в лекционной постановке используется чистое отношение.

25.1 Аренда самоката

Аренда стоит r в день, покупка — p , число дней неизвестно. Алгоритм арендует

$$K = \left\lceil \frac{p}{r} \right\rceil - 1$$

дней и перед следующим днём покупает. Если поездки прекратились раньше, он совпадает с оптимумом. Если срок больше:

$$\text{cost}_A = Kr + p < 2p, \quad \text{OPT} = p.$$

Следовательно, $\alpha < 2$. Если p/r целое, отношение равно

$$2 - \frac{r}{p},$$

и это оптимум среди детерминированных онлайн-алгоритмов: противник завершает последовательность непосредственно до или после момента покупки.

25.2 Кэширование

Кеш содержит k страниц; запрос отсутствующей страницы стоит 1 и требует вытеснения.

LRU является k -конкурентным, и ни один детерминированный алгоритм не имеет лучшего худшего коэффициента.

Marking. В начале фазы все метки сняты. Запрошенная страница помечается. При промахе вытесняется равномерно случайная непомеченная страница. Когда помечены k различных страниц, фаза заканчивается и метки снимаются.

Пусть m_i — число новых страниц в фазе i , то есть отсутствовавших в кеше в начале фазы. После запросов s различных старых страниц вероятность того, что следующая ещё не запрошенная старая страница уже вытеснена, не больше

$$\frac{m_i}{k - s}.$$

Поэтому ожидаемое число промахов фазы:

$$\mathbb{E}C_i \leq m_i \left(1 + \frac{1}{2} + \dots + \frac{1}{k}\right) = m_i H_k.$$

Для оптимального офлайн-алгоритма числа промахов n_i удовлетворяют

$$n_{i-1} + n_i \geq m_i,$$

откуда $\sum_i m_i \leq 2\text{OPT}$ с точностью до первой фазы. Следовательно,

$$\mathbb{E} \text{cost}_{\text{Marking}} \leq 2H_k \text{OPT} = O(\log k) \text{OPT}.$$

Эта оценка асимптотически оптимальна для вероятностных онлайн-алгоритмов.

Каркас ответа на экзамене

Определить competitive ratio. Для аренды разобрать два случая и получить < 2 . Для Marking определить фазу и метки, вывести гармоническую сумму промахов и сравнить $\sum m_i$ с офлайн-оптимумом.

Билет 26. Минимальное остовное дерево: Борувка, Прим и Краскал

Ключевая идея

Все три алгоритма поддерживают лес, содержащийся в некотором MST, и добавляют безопасные рёбра. Безопасность следует из свойства разреза.

Пусть $G = (V, E)$ — связный неориентированный граф с весами w . MST — остовное дерево минимальной суммарной стоимости.

Теорема 26.1 (Свойство разреза). Для любого разреза $(S, V \setminus S)$ ребро минимального веса, пересекающее разрез, содержится в некотором MST. Если минимум единственен, оно содержится в каждом MST.

Доказательство. Пусть MST T не содержит минимальное ребро e . В $T \cup \{e\}$ возникает цикл, содержащий другое ребро f того же разреза. Замена f на e сохраняет остовность и не увеличивает вес. При строгом минимуме вес уменьшается, противоречие. $\square$

Если все веса различны, MST единственно: применяют обмен к минимальному ребру симметрической разности двух предполагаемых MST.

26.1 Алгоритм Борувки

Для каждой компоненты текущего леса выбрать минимальное исходящее ребро и добавить все выбранные безопасные рёбра. Затем стянуть компоненты и повторить. Каждая новая компонента содержит хотя бы две старые, поэтому число компонент уменьшается минимум вдвое. Число фаз $\mathcal{O}(\log |V|)$; одна фаза просматривает все рёбра:

$$T = \mathcal{O}(|E| \log |V|).$$

Алгоритм естественно параллелится.

26.2 Алгоритм Ярника–Прима

Начать с одной вершины и поддерживать одно растущее дерево T . На каждом шаге добавить минимальное ребро разреза $(V(T), V \setminus V(T))$. Очередь с приоритетами хранит лучшие рёбра/ключи внешних вершин.

$$\mathcal{O}(|E| \log |V|)$$

с бинарной кучей, либо $\mathcal{O}(|E| + |V| \log |V|)$ с кучей Фибоначчи.

26.3 Алгоритм Краскала

Отсортировать рёбра по весу. Рассматривать их по возрастанию и добавлять ребро тогда и только тогда, когда его концы лежат в разных компонентах леса. Компоненты поддерживает DSU/Union–Find.

Корректность: полезное рассматриваемое ребро является минимальным на разрезе одной из его компонент, поскольку все более лёгкие рёбра уже рассмотрены; значит, оно безопасно.

С сортировкой и DSU с union by rank + path compression:

$$T = \mathcal{O}(|E| \log |E|) + \mathcal{O}(|E| \alpha(|V|)) = \mathcal{O}(|E| \log |V|).$$

Алгоритм	Основной выбор	Время
Борувка	минимальное исходящее ребро каждой компоненты	$\mathcal{O}(E \log V)$
Прим	минимальное ребро текущего разреза одного дерева	$\mathcal{O}(E \log V)$
Краскал	глобально следующее ребро, не создающее цикл	$\mathcal{O}(E \log V)$

Каркас ответа на экзамене

Сначала доказать свойство разреза обменом. Затем для каждого алгоритма указать инвариант “лес содержится в MST”, правило выбора безопасного ребра, структуру данных и оценку времени.

Сводная таблица оценок

Задача/алгоритм	Работа/время	Дополнительная характеристика
Карацуба	$\mathcal{O}(n^{\log_2 3})$	рекурсивное умножение
FFT и DFT-умножение	$\Theta(n \log n)$	глубина FFT $\Theta(\log n)$
Odd-even / bitonic sort	$\Theta(n \log^2 n)$	глубина $\Theta(\log^2 n)$
Ладнер–Фишер	размер $2n - 2$	глубина $2 \log_2 n$
Префикс на BSP	comp, comm $\mathcal{O}(n/p)$	супс $\mathcal{O}(1)$ при $n \geq p^2$
Carry lookahead	размер $\mathcal{O}(n)$	глубина $\mathcal{O}(\log n)$
Форд–Фалкерсон, целые ёмкости	$\mathcal{O}(E f^*)$	псевдополиномиально
Vertex cover	коэффициент 2	greedy/LP/primal-dual
Double-tree / Christofides	2 / 3/2	metric TSP
Фрейвальдс	$\mathcal{O}(kn^2)$	ошибка $\leq 2^{-k}$
Schwartz–Zippel	$\mathcal{O}(kT_p)$	ошибка $\leq (d/ S)^k$
FKS perfect hashing	$\mathcal{O}(s)$ память	поиск $\mathcal{O}(1)$ worst-case
2-SAT SCC	$\mathcal{O}(n + m)$	детерминированно
2-SAT random walk	$\mathcal{O}(mn^2)$	успех $\geq 1/2$
3-SAT Schönig	$\mathcal{O}^*((4/3)^n)$	односторонняя ошибка
Miller–Rabin	$\mathcal{O}(k \log^3 N)$	ошибка экспоненциально мала
Marking cache	$\mathcal{O}(\log k)$ -competitive	в ожидании
MST algorithms	$\mathcal{O}(E \log V)$	Boruvka/Prim/Kruskal

Источник и границы конспекта

Основой послужили лекции А. В. Тискина “Математические основы алгоритмов” (СПбГУ, весна 2026). Формулировки и обозначения сохранены максимально близко к лекционным; доказательства упражнений развёрнуты, а некоторые стандартные детали добавлены для самодостаточности. Для сильной двойственности приведён лекционный геометрический аргумент в невырожденном случае с указанием стандартного перехода к общему случаю.